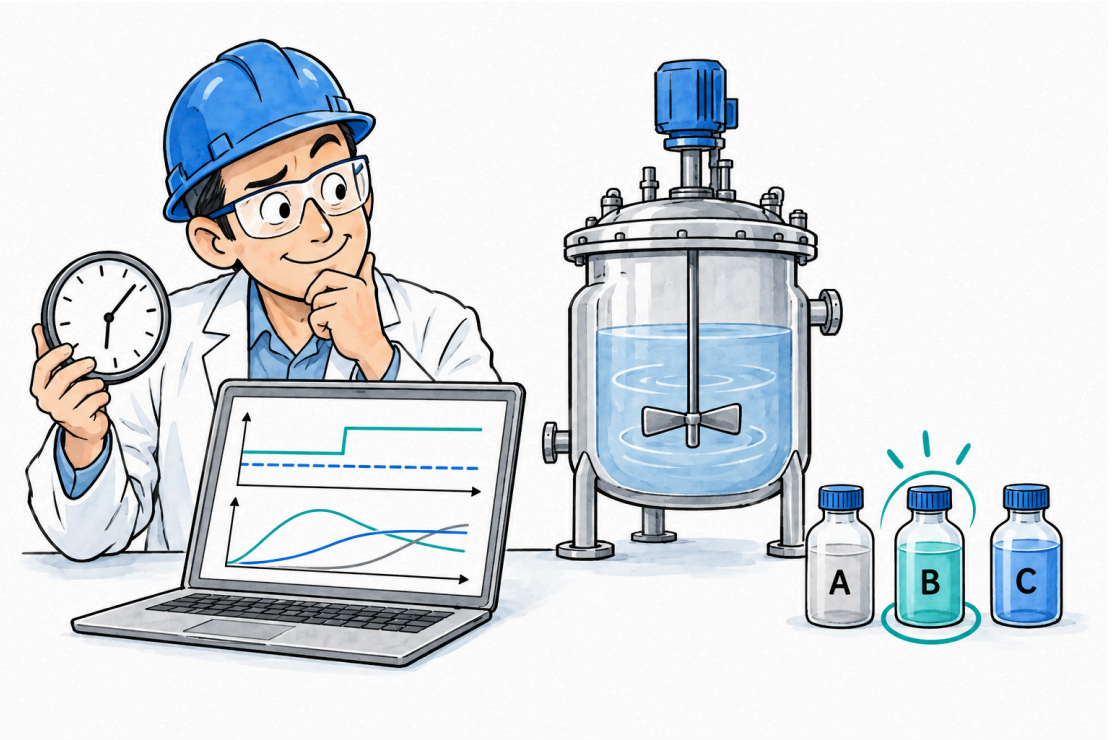
Week 2: Learning Multicomponent Steady-State Responses with MLPs

More conversion does not tell the whole composition

The desired intermediate is B in A → B → C. At short residence time little B forms; at long residence time more B reacts onward to C.

An engineer must predict all three outlet concentrations at each condition because conversion alone does not determine product composition.

Can one model predict A, B, and C consistently?



Three outputs, different errors

All three concentrations have the same units, but their variation differs. An unscaled loss can emphasize one component.

This week we calculate MLP outputs, follow one gradient update, and evaluate each component.

Learn and check the steady-state MLP

1. Derive steady-state labels for A, B, and C.
2. Scale the data and follow a forward pass, backpropagation, and one parameter update.
3. Train the MLP and compare component RMSE and concentration-sum residuals.

1. A → B → C in a CSTR

Consider a perfectly mixed, constant-volume, constant-density, isothermal CSTR with consecutive first-order reactions A → B → C. The feed contains A; its B and C concentrations are zero. Temperature is an operating input. With concentrations in mol/L, temperature in K, and time in min, the balances are:

$$\frac{dC_A}{dt} = \frac{C_{Af} - C_A}{\tau} - k_1 C_A$$

$$\frac{dC_B}{dt} = -\frac{C_B}{\tau} + k_1 C_A - k_2 C_B$$

$$\frac{dC_C}{dt} = -\frac{C_C}{\tau} + k_2 C_B$$

Rate constants and operating domain

Both rate constants have units of 1/min. Use the following temperature dependence:

$$k_1\left(T\right) = 0.2\exp\left[6000\left(\frac{1}{330} - \frac{1}{T}\right)\right]$$
$$k_2\left(T\right) = 0.1\exp\left[5000\left(\frac{1}{330} - \frac{1}{T}\right)\right]$$

Quantity	Value or range
Reference temperature	330 K
Reference k_1; k_2	0.2; 0.1 per minute
Temperature-sensitivity parameters	6000 K; 5000 K
Temperature T	300–360 K
Residence time τ	1–10 min
Feed A concentration	0.8–1.5 mol/L

The three balances describe A consumption, B formation and consumption, and C formation. Adding them cancels the reaction terms.

2. Derive the three-output steady-state map

Set the time derivatives to zero and multiply each balance by τ. Collect the concentration terms:

$$\left(1 + k_1\tau\right) C_A = C_{\mathrm{Af}}$$
$$\left(1 + k_2\tau\right) C_B = k_1\tau C_A$$
$$C_C = k_2\tau C_B$$

Solve for A, B, and C

Solve in the order A, B, C:

$$C_A = \frac{C_{\mathrm{Af}}}{1 + k_1\tau}$$

$$C_B = \frac{k_1\tau C_{\mathrm{Af}}}{(1 + k_1\tau)(1 + k_2\tau)}$$

$$C_C = k_2\tau C_B$$

At 330 K, τ = 5 min, and feed = 1.2 mol/L, the concentrations are [0.6, 0.4, 0.2] mol/L. Their sum equals the feed concentration. These three outputs are the labels for the steady-state MLP.

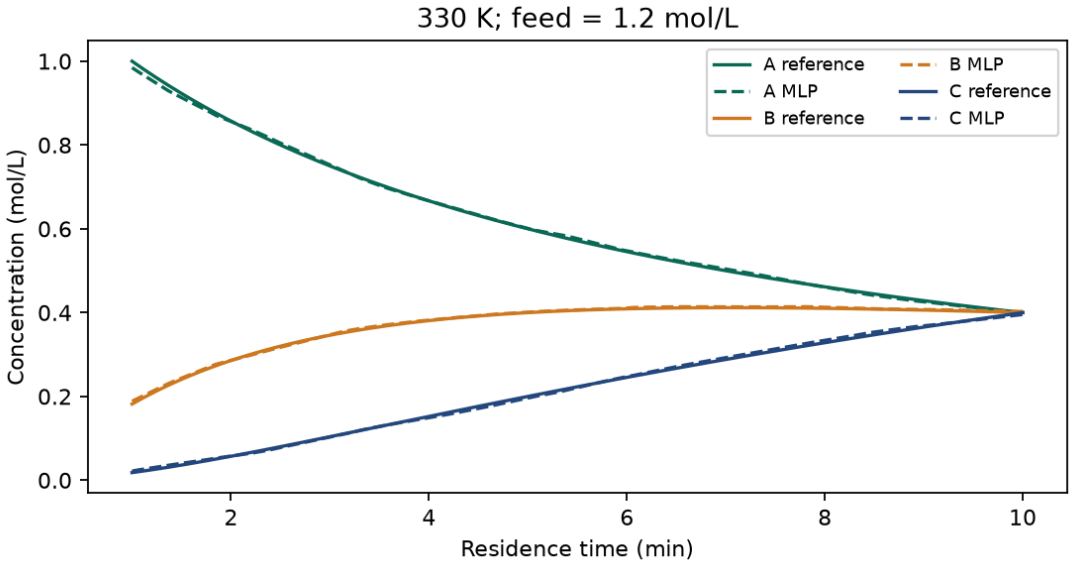
Why the B response is nonmonotone

At a fixed temperature, the B yield and its maximizing residence time follow from the expression for C_B:

$$Y_B = \frac{C_B}{C_{\mathrm{Af}}} = \frac{k_1\tau}{\left(1 + k_1\tau\right)\left(1 + k_2\tau\right)}, \quad \tau^* = \frac{1}{\sqrt{k_1 k_2}}$$

Compare the three concentration profiles

At 330 K, τ* is about 7.07 min. Short residence times limit B formation; long residence times allow more B to react to C.



Three-component response at 330 K. The B concentration has an interior maximum.

3. Prepare the supervised dataset

Each row contains inputs [T, τ, C_Af] and outputs [C_A, C_B, C_C]. Input and output arrays both have shape (N, 3). Generate 400 train, 120 validation, and 160 test operating points within the table's bounds, then calculate their labels with the steady-state map.

The three concentrations have the same units but different variations. In the training data, their standard deviations are approximately [0.283, 0.110, 0.217] mol/L.

4. Forward calculation and scaling

Use a 3 → 32 → 32 → 3 MLP, with ReLU hidden layers and a linear output layer:

Each hidden layer first computes a weighted sum and bias. ReLU(a) = max(0, a) retains positive values and sets negative values to zero. The weights and biases are the parameters fitted to the concentration data.

$$h_1 = \mathrm{ReLU}\left(W_1 \tilde{\mathrm{x}} + b_1\right)$$
$$h_2 = \mathrm{ReLU}\left(W_2 h_1 + b_2\right)$$
$$\hat{z} = W_3 h_2 + b_3$$

Layer	Weight shape	Bias shape	Parameters
Input → hidden 1	32 × 3	32	128
Hidden 1 → hidden 2	32 × 32	32	1056
Hidden 2 → output	3 × 32	3	99
Total			**1283**

Scaling and the training loss

Scale the inputs with the declared lower and upper bounds. Standardize each output with its training mean μ and standard deviation s, and restore physical units after prediction:

$$\tilde{x}_j = 2\frac{x_j - l_j}{u_j - l_j} - 1$$

$$z_j = \frac{y_j - \mu_j}{s_j}, \quad \hat{y}_j = \mu_j + s_j \hat{z}_j$$

The loss averages squared errors across samples and standardized output components:

$$L = \frac{1}{3N} \sum_{n=1}^{N} \sum_{j=1}^{3} \left(\hat{z}_{n,j} - z_{n,j} \right)^2$$

Thus a concentration error is measured relative to that component's training variation. For example, the smaller B standard deviation gives its raw squared error a larger weight in this loss.

5. Backpropagation: one calculation

Take a 3 → 2 → 3 network with zero biases and the following values. For this calculation, use $L = \frac{1}{2}\|\hat{y} - y\|^2$ and learning rate 0.1.

$$x = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$W_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

The hidden pre-activation a, hidden activation h, prediction, and error are:

$$a = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, h = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \hat{y} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, e = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$$

Gradients and the parameter update

The output error propagates through the output weights and the ReLU derivative:

$$\delta_2 = e, \nabla_{W_2} L = eh^T = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$\delta_1 = \left({W_2}^T e \right) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

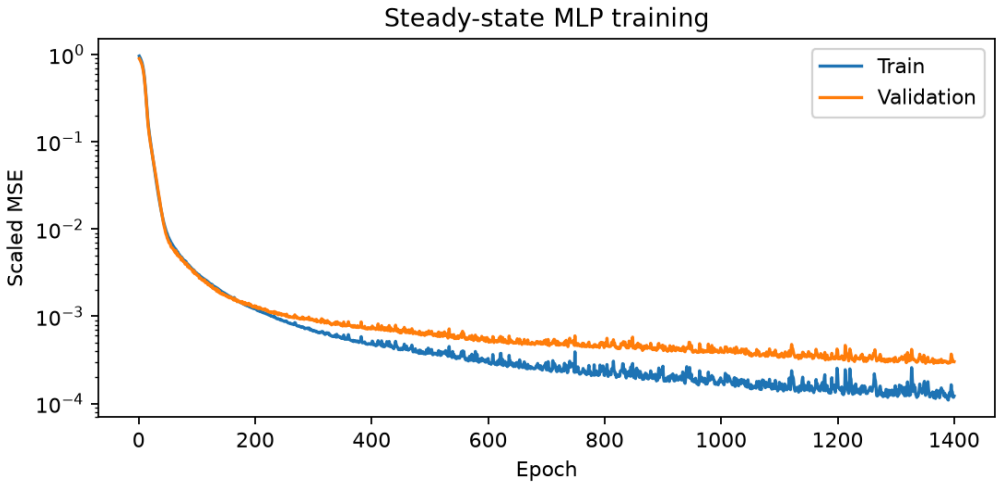
$$\nabla_{W_1} L = \delta_1 x^T = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}$$

The bias gradients are δ_1 and δ_2. Update every weight and bias with parameter ← parameter − 0.1 × gradient. The next forward calculation gives [0.26, 0.14, 0.26], and the loss decreases from 1.5 to 0.4374. The notebook checks these gradients using autograd.

6. Train and evaluate the steady-state model

Use Adam with learning rate 0.001, batches of 64, and up to 1400 epochs. After each epoch, evaluate validation loss and retain the weights at its minimum.

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

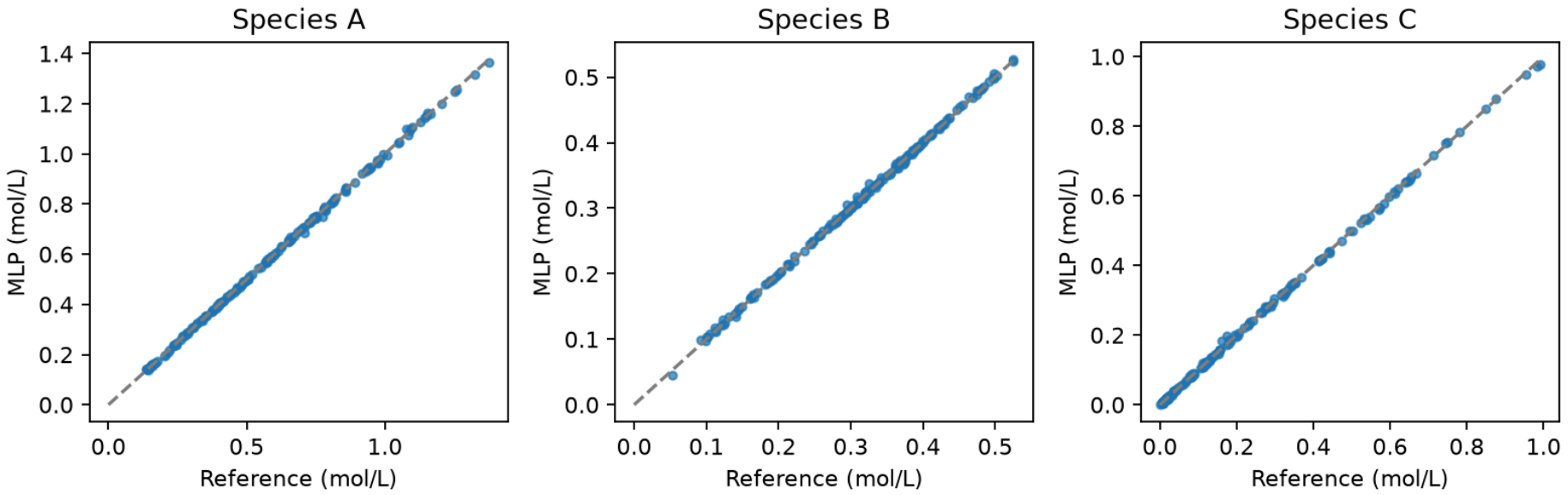


Training and validation loss. Prediction metrics below are computed after restoring mol/L units.

Evaluate each output component

Test RMSE, mol/L	A	B	C
Output scaling	0.005275	0.002940	0.004423
Unscaled outputs	0.006634	0.003917	0.004162

In this comparison, output scaling reduces A and B RMSE; C RMSE is slightly higher. Evaluate each component rather than combining all three into one error number.



Separate parity plots reveal each component's errors.

Check the concentration sum

$$r = \hat{C}_A + \hat{C}_B + \hat{C}_C - C_{\mathrm{Af}}$$

Its test RMSE is 0.003781 mol/L. Component prediction errors and the residual describe different aspects of the model.

Balance-consistent training labels do not force the sum of the three linear outputs to match the feed concentration.

For feed 1.2 mol/L, predictions [0.62, 0.41, 0.20] mol/L sum to 1.23 mol/L, giving r = 0.03 mol/L. A model predicting mole fractions can likewise return a sum different from one.

Week 4 preview: put the constraint into the model

We can correct a prediction after training, or include a constraint-respecting transformation in the training calculation.

Approach	Where the correction enters
Post-training mapping/projection	Train the raw predictor, then correct its outputs.
KKT-hPINN	Apply a fixed affine projection using the input and raw prediction before the loss, and at inference.

$$x \rightarrow f_\theta \rightarrow \hat{y} \rightarrow \Pi_x \rightarrow \tilde{y} \rightarrow \text{loss}$$

KKT-hPINN computes the loss on projected outputs and backpropagates through the projection layer. Projection coefficients stay fixed while the MLP weights learn with the transformation in place.

The projection layer satisfies the specified linear equalities up to numerical arithmetic; nonnegativity needs an additional constraint.

Source: Chen et al. (2024), DOI: 10.1016/j.compchemeng.2024.108764.

7. Run the required steady-state lab

Download the steady-state notebook.

```
python -m pip install -r requirements.txt
python week02_lab.py --mode steady --output-dir week02-results
```

Evaluate the trained predictor by component RMSE and concentration-sum residual.

8. Core exercises and submission

1. Derive C_A, C_B, and C_C and verify their sum. Explain the nonmonotone B curve.
2. Count the 1283 steady-state MLP parameters and verify the hand-calculated gradient update.
3. Compare scaled and unscaled outputs by each component's RMSE.
4. Inspect concentration-sum residual and minimum predicted concentration on held-out inputs.

Submit the steady-state model, split/scaling details, checked gradient update, and component/consistency evaluation.

Original explanations: PyTorch model construction, autograd, and training.

Reading companion

The Week 2 companion and English PDF work through the three-output map, parameter count, scaling, gradient update, training, and independent evaluation.

KKT-hPINN reference: Chen, H., Constante Flores, G. E., & Li, C. (2024). Physics-informed neural networks with hard linear equality constraints. *Computers & Chemical Engineering*, 189, 108764.